Foundations of Machine Learning and Deep Learning

Recurrent Neural Networks

University Master's Degree in Machine Learning

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

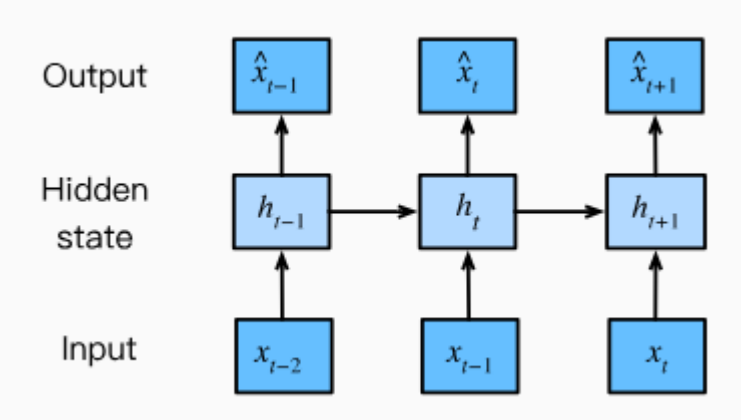
Autoregressive models

- **REMEMBER FROM YESTERDAY**
- Input size?
  - The number of inputs $x_{t-1}, \dots, x_1$ varies depending on t
- Possible solutions
  - Condition on some window of length τ and only use $x_{t-1}, \dots, x_{t-\tau}$ observations

$$\hat{x} = P(x_t | x_{t-1}, \dots, x_{t-\tau})$$

  - Mantain some summary h_t of the past observations and update it at every step. h_t is never observed → latent auteregressive model

$$\hat{x} = P(x_t | h_t)$$
$$h_t = g(h_{t-1}, x_{t-1})$$



RNN

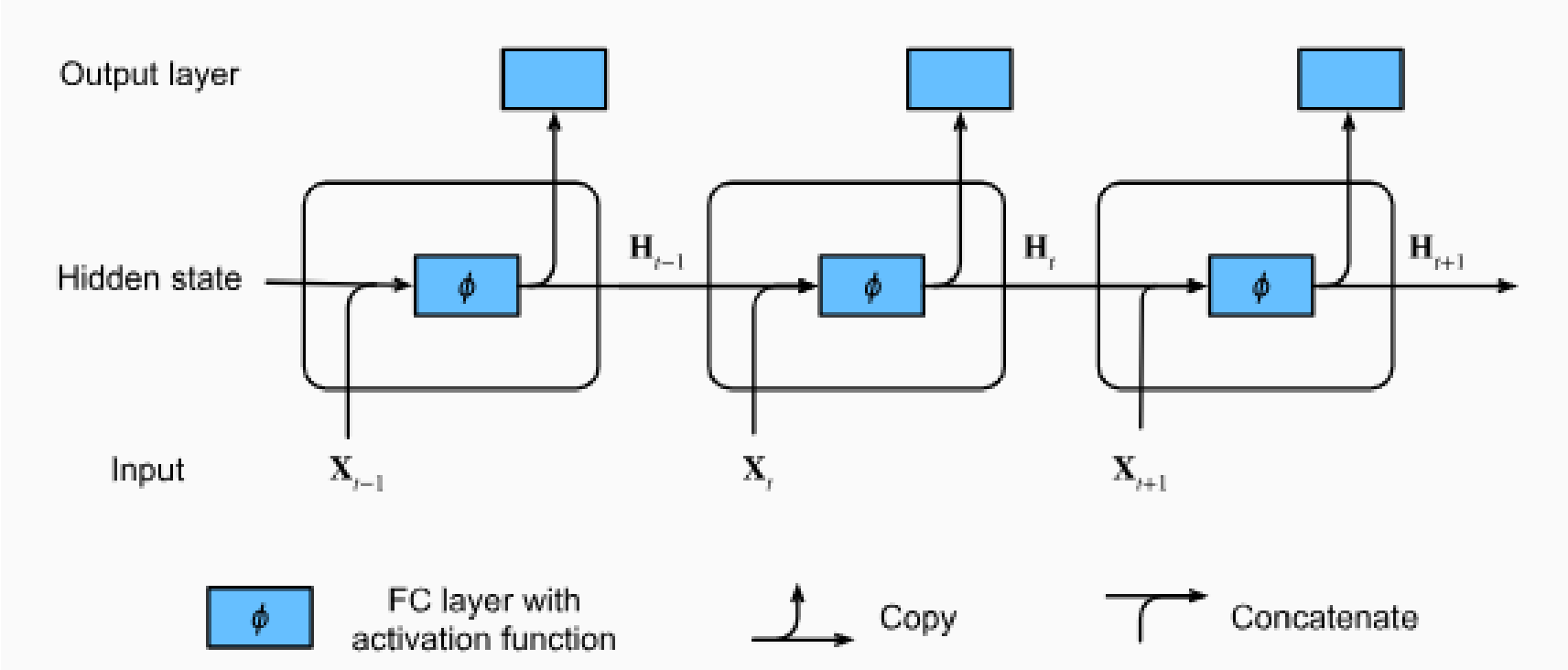
- Neural networks without hidden states (MLP with one hidden layer)

$$\boldsymbol{H} = \phi(\boldsymbol{X}\boldsymbol{W}_{xh} + \boldsymbol{b}_h)$$
$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}_{hq} + \boldsymbol{b}_q$$

  - Sizes
    - $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ (batch size n and d inputs)
    - $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$ $\quad$ $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$
    - $\boldsymbol{H} \in \mathbb{R}^{n \times h}$
    - $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ $\quad$ $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$
    - $\boldsymbol{O} \in \mathbb{R}^{n \times q}$

RNN

- Recurrent neural networks with hidden states



RNN

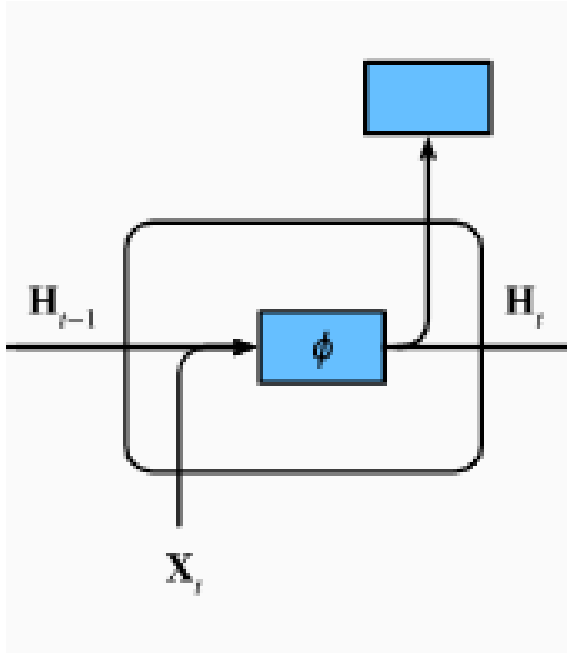
- Recurrent neural networks with hidden states

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$
$$\boldsymbol{O}_t = \boldsymbol{H}_t \boldsymbol{W}_{hq} + \boldsymbol{b}_q$$

- Sizes
  - $\boldsymbol{X_t} \in \mathbb{R}^{n \times d}$ (batch size n and d inputs at time t)
  - $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$ $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$ (h is the size of the hidden state)
  - $\boldsymbol{H}_{t-1}, \boldsymbol{H}_t \in \mathbb{R}^{n \times h}$
  - $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$
  - $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$
  - $\boldsymbol{O_t} \in \mathbb{R}^{n \times q}$
- The parametrization cost of an RNN does not grow as the number of time steps increases

RNN

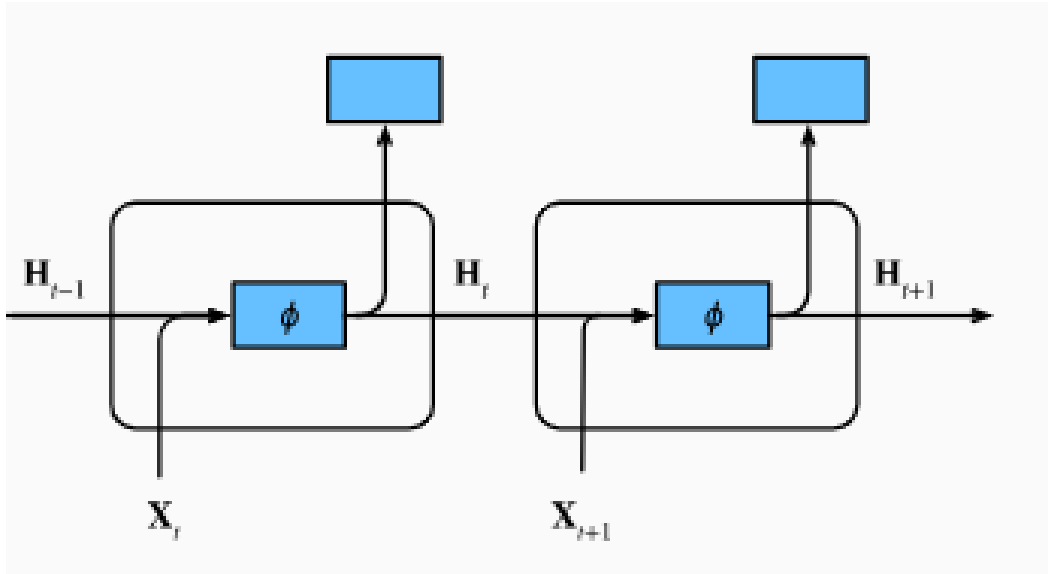
- Minibatch size=2
  - Sequence 1: ABC
    - $X_0 = A \rightarrow loss_1 = 0 + loss(B, O_0)$
    - $X_1 = B \rightarrow loss_1 = loss_1 + loss(C, O_1)$
  - Sequence 2: CBA
    - $X_0 = C \rightarrow loss_2 = 0 + loss(B, O_0)$
    - $X_1 = B \rightarrow loss_2 = loss_2 + loss(A, O_1)$
  - $loss_{batch} = loss_1 + loss_2$
  - $backward(loss_{batch})$
  - Update parameters



Backpropagation through time

- **Example**
  - Sequence of length 2
  - For simplicity: no bias and activation function $f(x) = x$

  - $h_0 = 0$
  - $h_1 = W_{xh}x_1 + W_{hh}h_0$
  - $y_1 = W_{hq}h_1$
  - $L_1 = \frac{1}{2}(y - \hat{y})^2$
  - $h_2 = W_{xh}x_2 + W_{hh}h_1$
  - $y_2 = W_{hq}h_2$
  - $L_2 = \frac{1}{2}(y - \hat{y})$
  - $L = L_1 + L_2$



Backpropagation through time

- Gradient calculus
- W_{hq}
- $\frac{\partial L}{\partial W_{hq}} = \frac{\partial L_1}{\partial W_{hq}} + \frac{\partial L_2}{\partial W_{hq}} = \frac{\partial L_1}{\partial y_1}\frac{\partial y_1}{\partial W_{hq}} + \frac{\partial L_2}{\partial y_2}\frac{\partial y_2}{\partial W_{hq}} = (y_1 - \hat{y}_1)h_1 + (y_2 - \hat{y}_2)h_2$

- W_{hh}
- $\frac{\partial L}{\partial W_{hh}} = \frac{\partial L_1}{\partial W_{hh}} + \frac{\partial L_2}{\partial W_{hh}}$
- $\frac{\partial L_1}{\partial W_{hh}} = \frac{\partial L_1}{\partial y_1}\frac{\partial y_1}{\partial h_1}\frac{\partial h_1}{\partial W_{hh}} = (y_1 - \hat{y}_1)W_{hq}h_0 = 0$
- $\frac{\partial L_2}{\partial W_{hh}} = \frac{\partial L_2}{\partial y_2}\frac{\partial y_2}{\partial h_2}\left[\frac{\partial h_2}{\partial W_{hh}} + \frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W_{hh}}\right] = (y_2 - \hat{y}_2)W_{hq}[h_1 + W_{hh}h_0] = (y_2 - \hat{y}_2)W_{hq}h_1$

Backpropagation through time

- Gradient calculus
- W_{xh}
- $\frac{\partial L}{\partial W_{xh}} = \frac{\partial L_1}{\partial W_{xh}} + \frac{\partial L_2}{\partial W_{xh}}$
- $\frac{\partial L_1}{\partial W_{xh}} = \frac{\partial L_1}{\partial y_1}\frac{\partial y_1}{\partial h_1}\frac{\partial h_1}{\partial W_{xh}} = (y_1 - \hat{y}_1)W_{hq}x_1$
- $\frac{\partial L_2}{\partial W_{xh}} = \frac{\partial L_2}{\partial y_2}\frac{\partial y_2}{\partial h_2}\left[\frac{\partial h_2}{\partial W_{xh}} + \frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W_{xh}}\right] = (y_2 - \hat{y}_2)W_{hq}[x_2 + W_{hh}x_1]$
- $\frac{\partial L}{\partial W_{xh}} = (y_1 - \hat{y}_1)W_{hq}x_1 + (y_2 - \hat{y}_2)W_{hq}[x_2 + W_{hh}x_1]$

Backpropagation through time

- General term

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} (y_t - \hat{y}_t) W_{hq} \left(\sum_{k=1}^{t} W_{hh}^{t-k} x_k \right)$$

- The loss at time step t depends on the loss at previous steps
- Gradients are accumulated using the chain rule
- If the sequences are long, we have lots of matrix multiplications
  - High storing complexity
  - Vanishing gradients
  - Exploding gradients

Backpropagation through time

- Vanishing gradients: if the recurrent weight matrix has eigenvalues < 1, gradients shrink exponentially as they propagate backward. This makes it very hard for the model to learn long-term dependencies
- Exploding gradients: if eigenvalues > 1, gradients grow exponentially, leading to instability

- Gradient clipping: Before performing the parameter update, check the norm of the gradients
- If the norm exceeds a predefined threshold, rescale the gradients so that their norm equals the threshold
- torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm)
- Truncated BPTT (backpropagate just k steps)

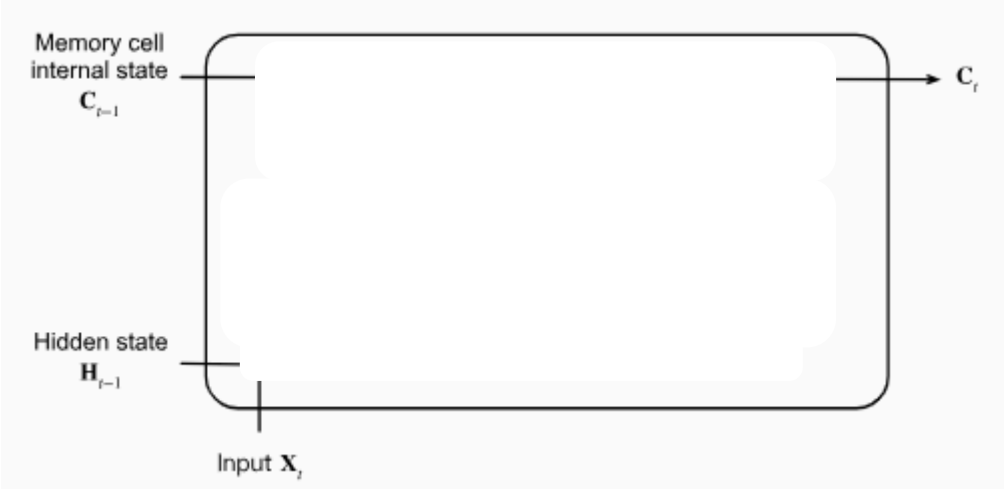
Modern recurrent neural networks

- Introduce gating mechanisms to control the flow of information through time
- Allow the model to retain long-term dependencies more effectively

- LSTM: Long Short-Term Memory networks
- GRU: Gated Recurrent Units

LSTM

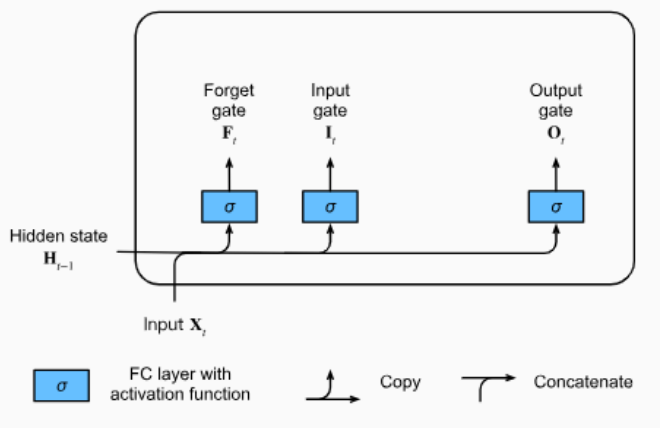
- Besides the hidden state, introduces a cell state like a memory vector that can carry information across many timesteps with minimal modification



LSTM

- Gates to interact with the memory cell internal state
  - Forget gate: degree of keeping the current value of the memory cell internal state
  - Input gate: how much of the input node's value should be added to the current memory cell internal state
  - Output gate: how much the memory cell influences the output at the current timestep

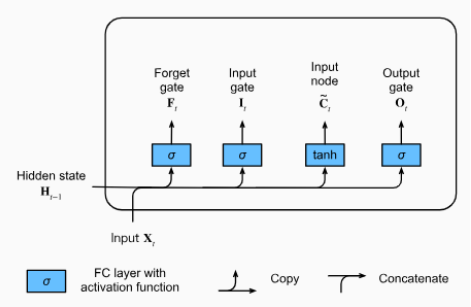
  - All of them are fully connected layers with sigmoid activation functions (the values of the gates are in the range (0, 1))



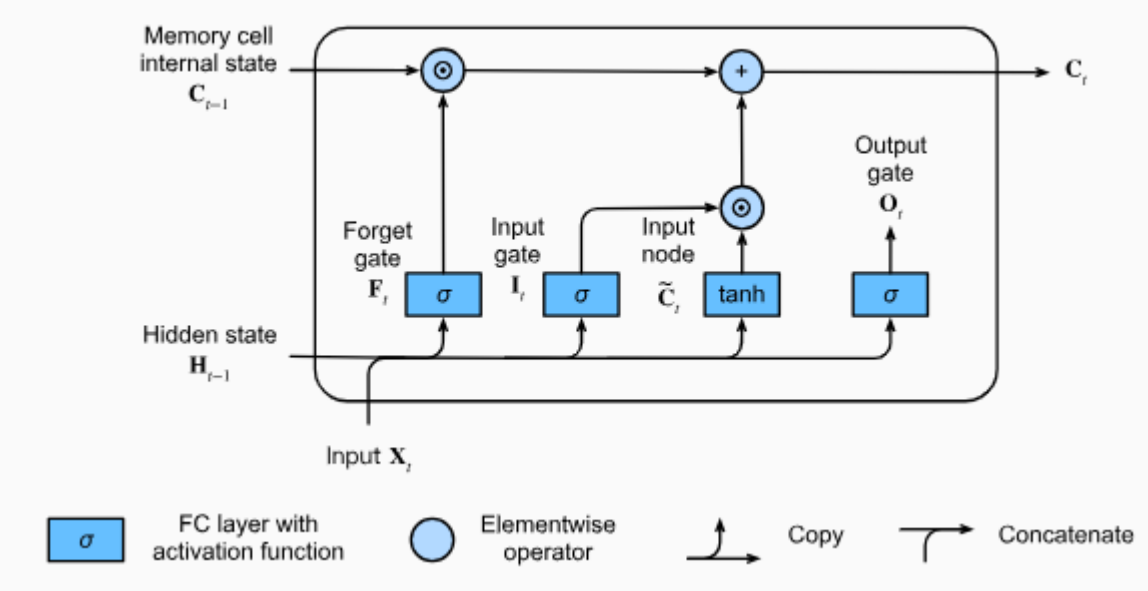
LSTM

- Input node: represents the new candidate information to be added to the memory

- It is a fully connected layer with a tanh activation function (values range for (-1, 1)

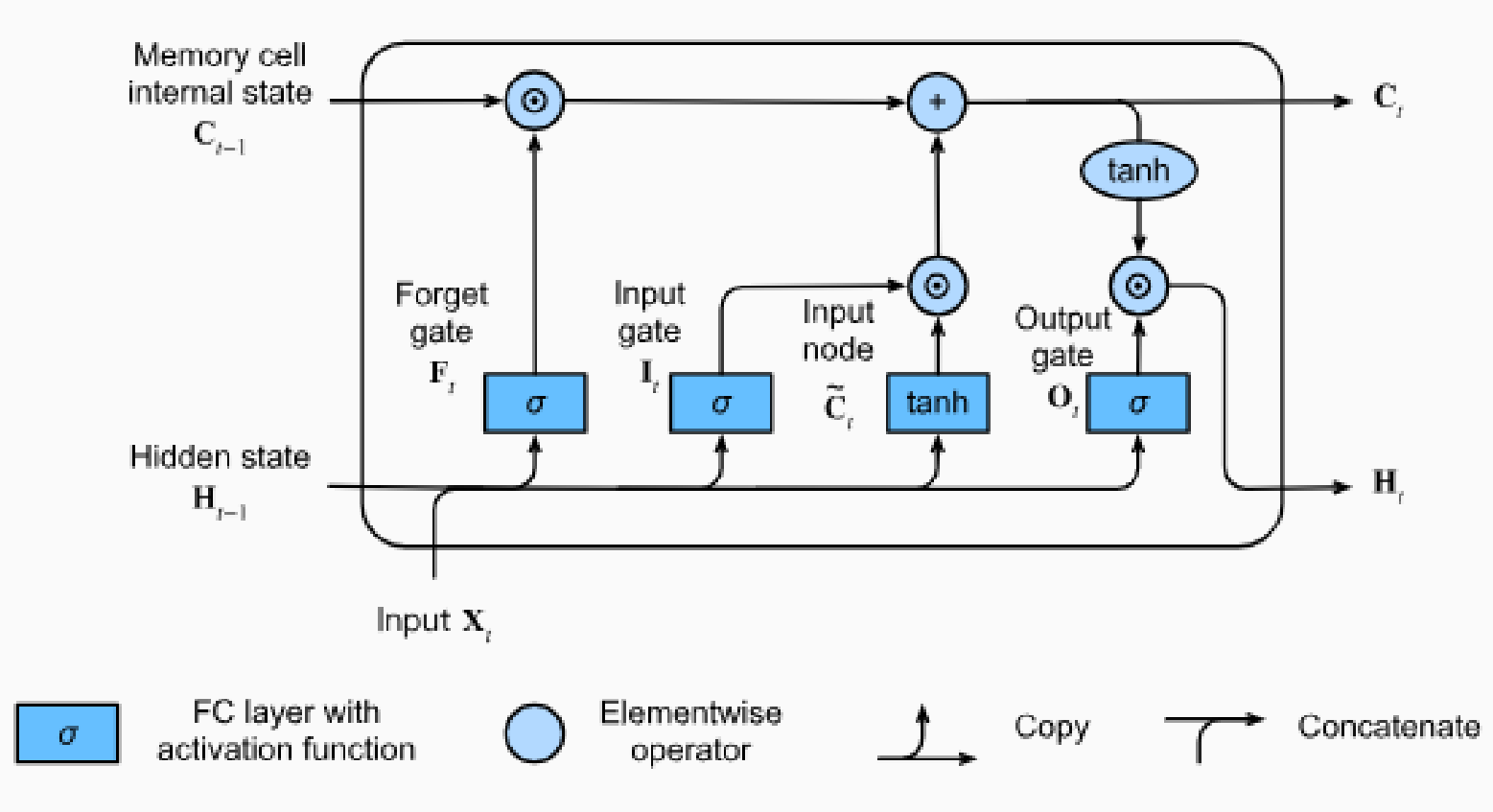


- Interaction with the memory cell internal state (⊙ is elementwise product)



LSTM

- To calculate the output of the cell (the hidden state), apply a tanh over the memory cell internal state and operate (elementwise product) with the output gate

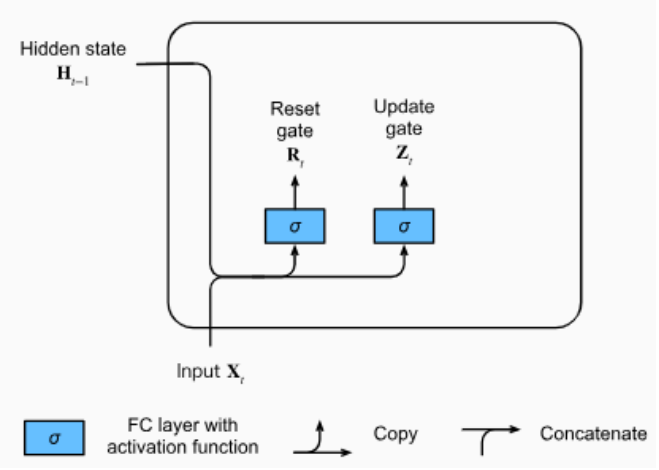


LSTM

- $\boldsymbol{F}_i = \sigma(\boldsymbol{X}_t\boldsymbol{W}_{xf} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hf} + \boldsymbol{b}_f)$
- $\boldsymbol{I}_t = \sigma(\boldsymbol{X}_t\boldsymbol{W}_{xi} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hi} + \boldsymbol{b}_i)$
- $\boldsymbol{O}_t = \sigma(\boldsymbol{X}_t\boldsymbol{W}_{xo} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{ho} + \boldsymbol{b}_o)$
- $\widetilde{\boldsymbol{C}}_t = tanh(\boldsymbol{X}_t\boldsymbol{W}_{xc} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hc} + \boldsymbol{b}_c)$
- $\boldsymbol{C}_t = \boldsymbol{F}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{I}_t \odot \widetilde{\boldsymbol{C}}_t$
- $\boldsymbol{H}_t = \boldsymbol{O}_t \odot \tanh(\boldsymbol{C}_t)$
- If there are h hidden units, the batch size is n, and the number of inputs is d
  - $\boldsymbol{X}_t \in \mathbb{R}^{n\times d};\ \boldsymbol{H}_t \in \mathbb{R}^{n\times h};\ \boldsymbol{C}_t \in \mathbb{R}^{n\times h}$
  - $\boldsymbol{F}_t, \boldsymbol{I}_t, \boldsymbol{O}_t \in \mathbb{R}^{n\times h}$
  - $\boldsymbol{W}_{xf}, \boldsymbol{W}_{xi}, \boldsymbol{W}_{xo} \in \mathbb{R}^{d\times h};\ \boldsymbol{W}_{hf}, \boldsymbol{W}_{hi}, \boldsymbol{W}_{ho} \in \mathbb{R}^{h\times h};\ \ \boldsymbol{b}_f, \boldsymbol{b}_i, \boldsymbol{b}_o \in \mathbb{R}^{1\times h}$
  - $\boldsymbol{W}_{xc} \in \mathbb{R}^{d\times h};\ \ \boldsymbol{W}_{hc} \in \mathbb{R}^{h\times h};\ \ \boldsymbol{b}_c \in \mathbb{R}^{1\times h}$

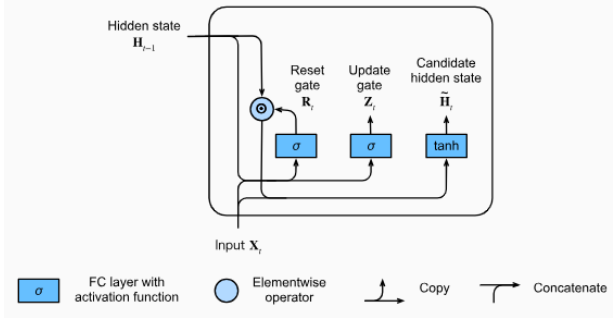
GRU

- The GRU simplifies the LSTM design while retaining the benefits of gating mechanisms
- GRU combine the hidden and cell states into a single vector, making them more lightweight
- Two fully connected layers with a sigmoid activation function
  - Reset gate: how much of the previous state we might still want to remember
  - Update gate: how mucho of the new state is just a copy of the old one
  - $\boldsymbol{R}_t = \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hr} + \boldsymbol{b}_r)$
  - $\boldsymbol{Z}_t = \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hz} + \boldsymbol{b}_z)$

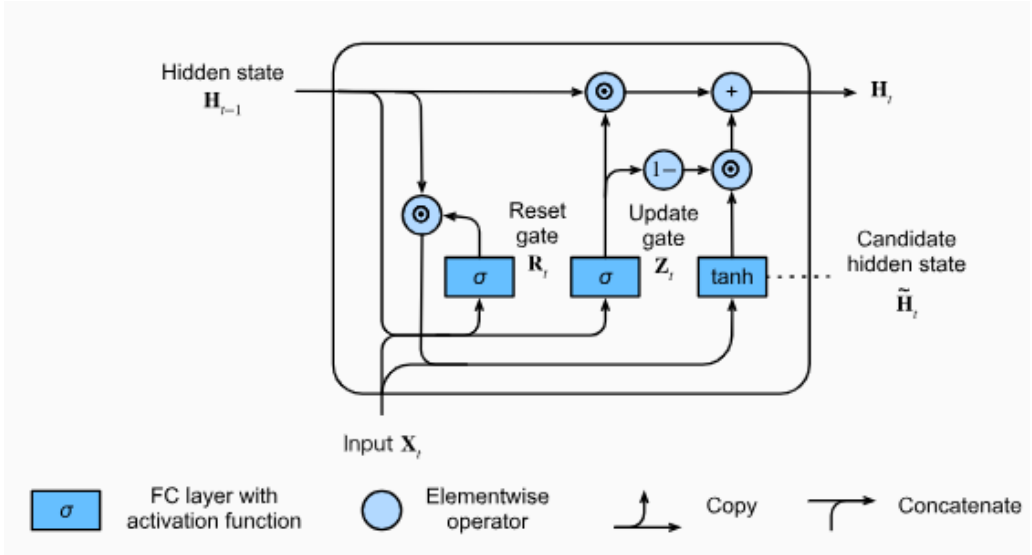


GRU

- Integrate reset gate with the regular updating mechanism to the candidate hidden state (if R_t is 1, we recover the RNN)
  - $\tilde{\boldsymbol{H}}_t = tanh(\boldsymbol{X}_t \boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$



- The update gate determines how much the hidden state matches the old hidden sate compared to how much it resembles the new candidate state (if Z_t is close to 1, ignore the input information. If it is close to 0, the hidden state equals the actual candidate)
  - $H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$



Comparison between RNN, LSTM and GRU

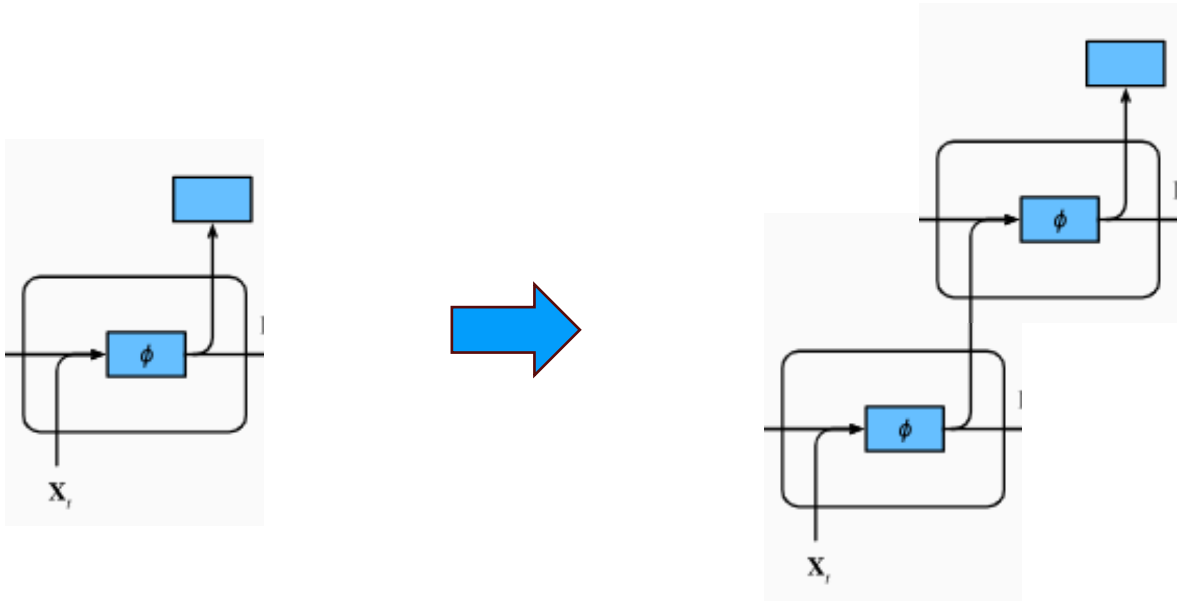
Feature	RNN	LSTM	GRU
Hidden state only	Yes	No	Yes
Gating mechanisms	No	3 gates (forget, input and output)	2 gates (update and reset)
Handles long-term memory	Poorly	Effectively	Effectively
Computational cost	Low	Higher (more parameters)	Medium (fewer parameters)
Gradient stability	Unstable	Stable	Stable

LSTM and GRU

- Maintain long-range information through controlled gates
  - The updates of the cell state (LSTM) and hidden state (GRU) are dome by adding, so prevent exponential shrinkage of gradients
- Exploding gradients can still occur → Gradient clipping is still necessary

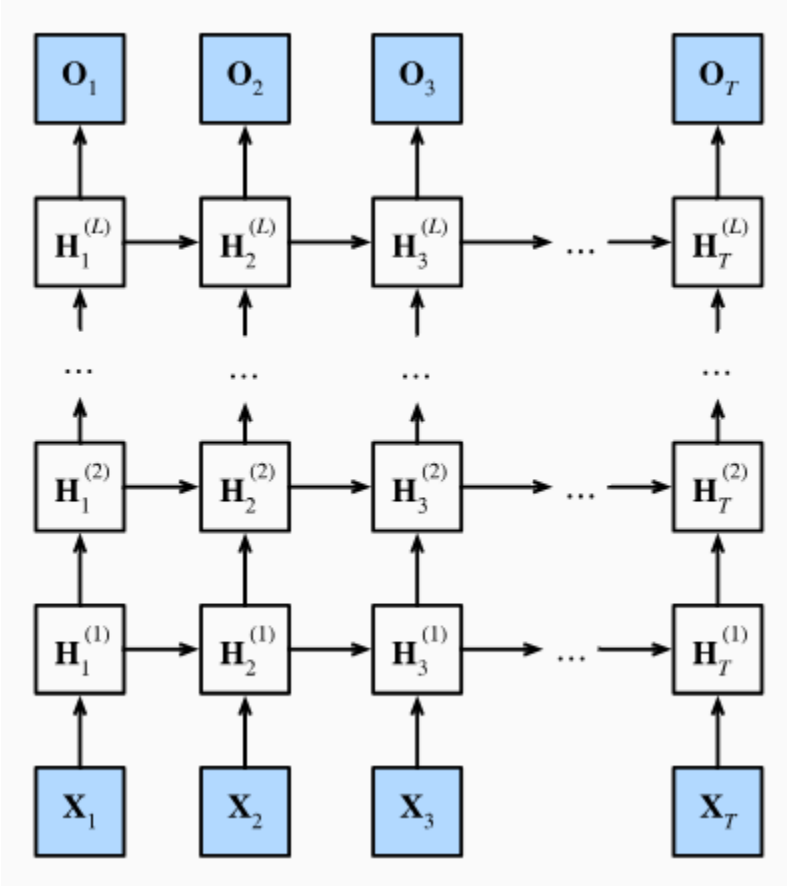
Deep recurrent neural networks

- Recurrent neural networks are deep in the time direction
  - Inputs from the first time step can influence outputs at the final time step
- What about the depth in the in the usual way?
  - Complex relations between the inputs at a given time step and the outputs at the same time step



Deep recurrent neural networks

- Stack RNNs on top of each other



Deep recurrent neural networks

- Minibatch input at time step t: $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (n examples and d dimensions)
- l^{th} hidden layer: $\boldsymbol{H}_t^{(l)} \in \mathbb{R}^{n \times h}$ (h number of hidden units)
- Output layer: $\boldsymbol{O}_t \in \mathbb{R}^{n \times q}$ (q number of outputs)
- Set $\boldsymbol{H}_t^{(0)} = \boldsymbol{X}_t$

$$\boldsymbol{H}_t^{(l)} = \phi(\boldsymbol{H}_t^{(l-1)}\boldsymbol{W}_{xh}^{(l)} + \boldsymbol{H}_{t-1}^{(l)}\boldsymbol{W}_{hh}^{(l)} + \boldsymbol{b}_h^{(l)}$$

$$\boldsymbol{O}_t = \boldsymbol{H}_t^{(l)}\boldsymbol{W}_{hq} + \boldsymbol{b}_q$$

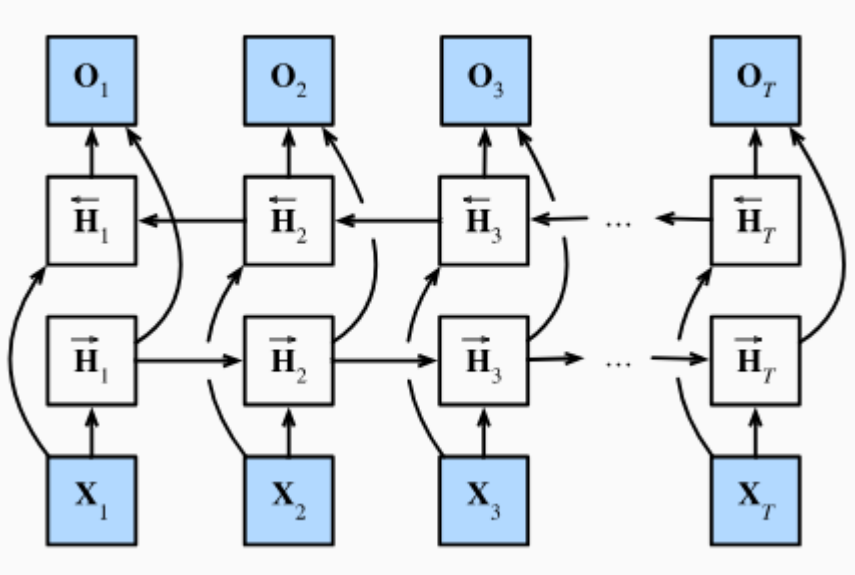
- $\boldsymbol{W}_{xh}^{(l)}, \boldsymbol{W}_{hh}^{(l)} \in \mathbb{R}^{h \times h}$; $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$
- $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$; $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$
- Common hiperparameter values:
  - Number of hidden units (h) → range (64, 2056)
  - Number of layers ($L; l = 1, \ldots, L$) → range (1,8)

Bidirectional recurrent neural networks

- Sometimes it is better to condition the prediction at every time step on both the leftward and rightward context
- Example: prediction of a missing token
  - I am _____
  - I am _____ hungry
  - I am _____hungry, and I can eat half a pig

Bidirectional recurrent neural networks

- Two unidirectional RNN layers in opposite directions
  - First RNN: input is $x_1 \dots x_T$
  - Second RNN: input is $x_T \dots x_1$
  - Concatenate the outputs to create the input to the output layer



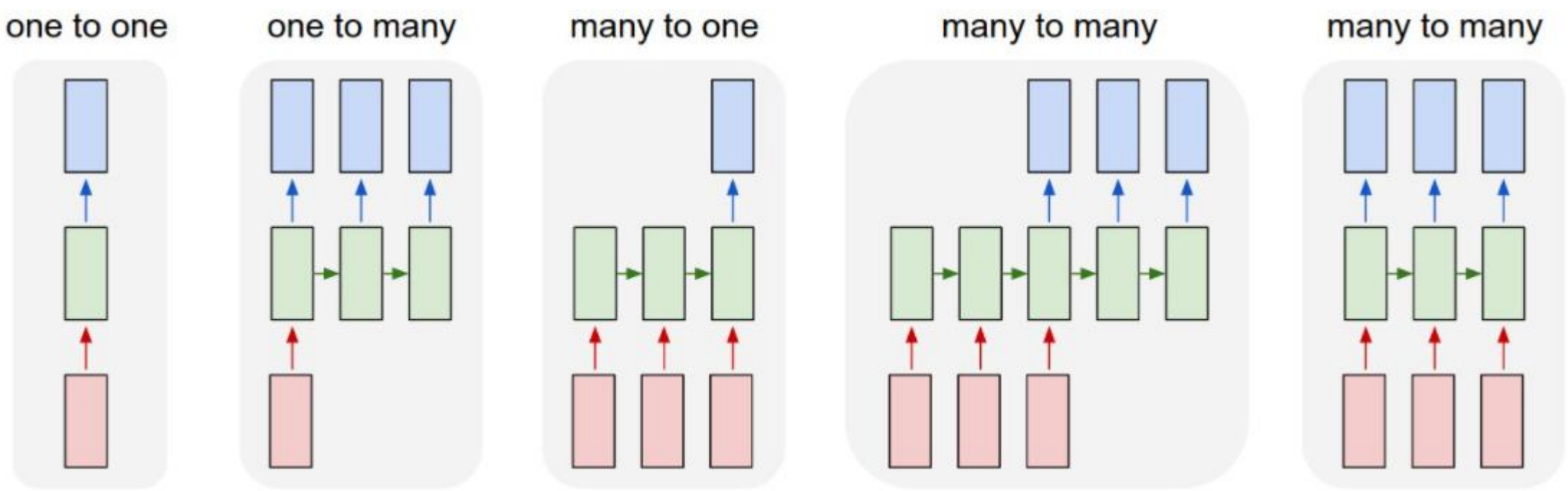
Bidirectional recurrent neural networks

- Minibatch input at time step t: $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (n examples and d dimensions)
- Hidden states: $\overrightarrow{\boldsymbol{H}_t}, \overleftarrow{\boldsymbol{H}_t} \in \mathbb{R}^{n \times h}$ (h number of hidden units)

$$\overrightarrow{\boldsymbol{H}_t} = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(f)} + \overrightarrow{\boldsymbol{H}_{t-1}} \boldsymbol{W}_{hh}^{(f)} + \boldsymbol{b}_h^{(f)})$$
$$\overleftarrow{\boldsymbol{H}_t} = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(b)} + \overleftarrow{\boldsymbol{H}_{t-1}} \boldsymbol{W}_{hh}^{(b)} + \boldsymbol{b}_h^{(b)})$$
$$\boldsymbol{H}_t = [\overrightarrow{\boldsymbol{H}_t} \; \overleftarrow{\boldsymbol{H}_t}]$$
$$\boldsymbol{O}_t = \boldsymbol{H}_t \boldsymbol{W}_{hq} + \boldsymbol{b}_q$$

- $\boldsymbol{W}_{xh}^{(f)}, \boldsymbol{W}_{xh}^{(b)} \in \mathbb{R}^{d \times h}$; $\boldsymbol{W}_{hh}^{(f)}, \boldsymbol{W}_{hh}^{(b)} \in \mathbb{R}^{h \times h}$; $\boldsymbol{b}_h^{(f)}, \boldsymbol{b}_h^{(b)} \in \mathbb{R}^{1 \times h}$
- $\boldsymbol{H}_t \in \mathbb{R}^{n \times 2h}$
- $\boldsymbol{O}_t \in \mathbb{R}^{n \times q}$
- $\boldsymbol{W}_{hq} \in \mathbb{R}^{2h \times q}$; $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$

Types of problems for RNNs



Types of problems for RNNs

- One to many
  - Image captioning (CNN→feature vector→RNN to describe)
- Many to one
  - Sentiment análisis
  - Detection of intention of a complete message
  - Fraud detection based on a sequence of transactions
- Many to many synchronus
  - Part of speech tagging
  - Transcription
- Many to many asynchronus
  - Translation
  - Summary of a text

Seq2seq

- Many to many asynchronus is known as seq2seq
  - Inputs and outputs are of varying lengths
  - Inputs and outputs are unaligned

- Encoder-decoder architecture
  - Encoder: takes a variable-length sequence as input and transforms it into a state with a fixed shape
  - Decoder: maps the encoded state of a fixed shape to a variable-length sequence

